



Tools and Session State

Giving Agents Memory and Actions

A practical guide for developers integrating ADK agents — covering how to register tools, manage in-memory session state, handle errors, and test multi-turn conversations end to end.

Before We Begin

Make sure you're comfortable with these foundational concepts before diving in. This session builds directly on them.

1

Python Functions

A named, reusable block of code that takes inputs and returns a value. Tools in ADK are standard Python functions. In modern ADK, plain Python functions added to the `tools=[]` list are automatically wrapped as `FunctionTool` — no decorator needed.

2

Type Hints & Docstrings

ADK inspects your function's type hints, parameter names, and docstring to generate the tool schema for the LLM. These are your primary documentation for the model.

3

Live Data Limitations

An agent cannot answer a real-time order question without a tool. It can only reference its prompt, training data, and current conversation.

4

Fallback Behavior

When the agent lacks an answer and has no tool to call, it should respond gracefully — not guess or crash.



QUESTION 1

What is a Tool in ADK, and Why Does an Agent Need One?

The Problem Without Tools

Without a tool, the agent is limited to what it already knows — its system prompt, training data, and the current conversation thread. It cannot fetch live data, query a database, or perform any real-world action.

The Solution: Tools

A **tool** is a Python function the agent can call to do real work — such as looking up an order status, checking stock levels, or triggering a cancellation.

A tool turns the agent from a *talker* into a *doer*. It bridges the gap between language understanding and real-world action.

How Does ADK Know a Function is a Tool?

You simply define a regular Python function with type hints and a clear docstring, then add it directly to the agent's `tools=[]` list.

ADK automatically wraps the function as a `FunctionTool` and generates the schema for the LLM from the function's name, docstring, parameters, and type hints.

```
from google.adk.agents import Agent

def get_order_status(order_id: str) -> dict:
    """Returns the current status and ETA for an order.

    Args:
        order_id (str): The unique order identifier.

    Returns:
        dict: Order status and estimated delivery date.
    """
    return {"status": "Shipped", "eta": "3 Jun 2026"}

agent = Agent(
    model="gemini-2.0-flash",
    name="order_agent",
    instruction="Help users track their orders.",
    tools=[get_order_status], # Functions are auto-wrapped as FunctionTool
)
```

- ❗ No decorator needed. ADK automatically wraps plain Python functions as `FunctionTool` when added to the tools list. The docstring is the primary signal the LLM uses to understand what the tool does — write it clearly and precisely.

QUESTION 3

How Does the Agent Decide When to Call a Tool?

The LLM decides autonomously, based on the user's request and the tool descriptions available to it. You do not hardcode routing logic — you make the tools and instructions clear enough for the model to choose correctly.

User Message	Agent Decision
"Hi, how are you?"	Respond directly from context
"Where is my order ORD-001?"	Call <code>get_order_status("ORD-001")</code>
"What is your return policy?"	Respond directly from prompt
"Cancel my order ORD-002"	Call <code>cancel_order("ORD-002")</code>

- 📌 The key insight: the model routes intelligently when tool names and docstrings are descriptive. Ambiguous naming leads to poor routing decisions.

How Do You Wire In-Memory Session State into ADK?

What Is Session State?

Session state is a **per-session key-value store** with strings as keys and serializable values. It lives only as long as the session when using `InMemorySessionService`, or it can persist across restarts with Database or Vertex AI session services.

Keys can use prefixes to define scope: no prefix means session-scoped, `app:` means app-wide, `user:` means per-user, and `temp:` means current invocation only and is discarded after the turn.

Inside tools, you access it through `tool_context.state`.

Reading and Writing State

```
from google.adk.tools import ToolContext

def my_tool(query: str, tool_context: ToolContext) -> dict:
    """A tool that reads and writes session state."""

    # Read from state (with safe default)
    user_name = tool_context.state.get("user_name", "there")

    # Write to state
    tool_context.state["user_name"] = "Priya"

    # Temp state: shared within one turn, discarded after
    tool_context.state["temp:last_query"] = query

    return {"greeting": f"Hello, {user_name}!"}
```

- ❑ ToolContext is injected automatically by ADK when the parameter is named `tool_context` with type `ToolContext`. No manual wiring required.

How Do You Use Session State Inside a Tool?

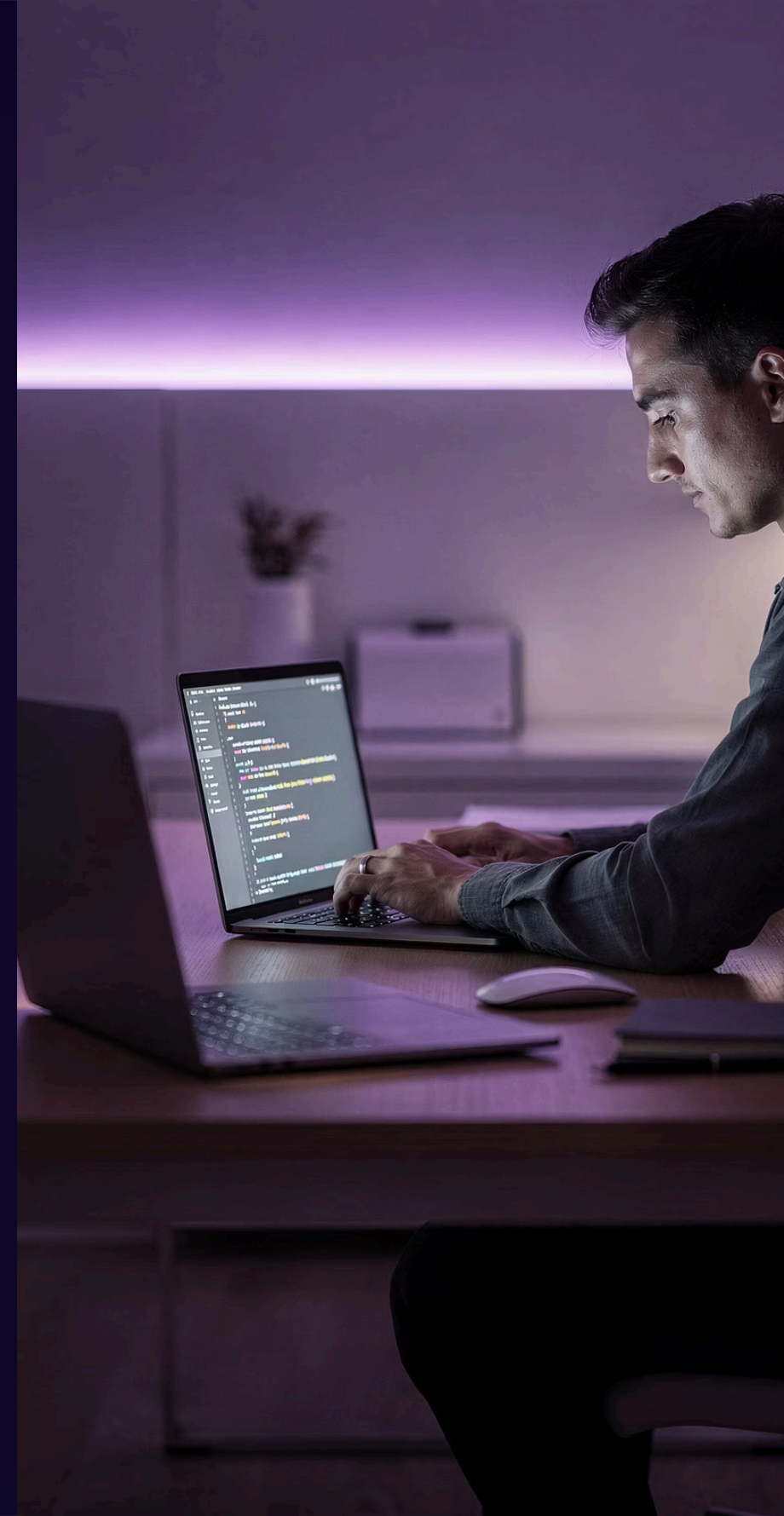
`ToolContext` is injected automatically by ADK when you declare a `tool_context`: `ToolContext` parameter. Read from state when you need prior context. Write to state when the agent learns something new. Use the `temp`: prefix to pass data between tools within the same turn.

```
from google.adk.tools import ToolContext

def greet_user(tool_context: ToolContext) -> dict:
    """Greet the user by their stored name."""
    # Read: use stored name in a personalized reply
    name = tool_context.state.get("user_name", "there")
    return {"message": f"Hello, {name}!"}

def save_user_name(name: str, tool_context: ToolContext) -> dict:
    """Saves the user's name to session state for future turns."""
    # Write: persist the name the user just told you
    tool_context.state["user_name"] = name
    return {"status": "saved", "name": name}
```

- ✔ `ToolContext` is injected automatically — just declare it as a typed parameter. If the user says "My name is Priya" in Turn 1, `save_user_name` stores it immediately. Every subsequent tool call can then read it from state, creating a coherent, personalized experience across the session.



How Do You Write a Tool That Returns Structured Output?

A tool should always return a **dictionary**, not a raw string. Structured output gives the agent clean, named fields to reference — making responses more reliable and the data ready for downstream UI rendering. If a function returns a non-dict type, ADK automatically wraps it in `{"result": value}`, but explicit dictionaries are clearer.

```
def get_order_status(order_id: str) -> dict:
    """Returns the current status and ETA for an order.

    Args:
        order_id (str): The unique order identifier.

    Returns:
        dict: A structured response with order details.
    """
    return {
        "order_id": order_id,
        "status": "Shipped",
        "eta": "3 Jun 2026",
        "carrier": "BlueDart"
    }
```

Why Dictionaries Win

Clear field names are far easier for the LLM to parse than a single long sentence. The agent can refer to `carrier` and `eta` individually, reducing hallucination risk. If a function returns a non-dict type, ADK will still wrap it as `{"result": value}`, but explicit dictionaries are the best practice.

Future-Proofing

Structured output also prepares your data for richer UI rendering later — displaying order cards, status badges, or timelines becomes straightforward when the fields are already defined.

What Happens When a Tool Fails?

Tools can fail due to bad input, missing records, or downstream service errors. Rather than raising exceptions, return a **clean error dictionary** — so the agent can respond politely and informatively instead of crashing.

```
def get_order_status(order_id: str) -> dict:
    """Returns the current status and ETA for an order.

    Args:
        order_id (str): The unique order identifier.
    """
    if not order_id.startswith("ORD-"):
        return {"error": "Invalid order ID format. Expected format: ORD-XXXX"}
    if order_id not in mock_orders:
        return {"error": f"Order {order_id} not found."}
    return mock_orders[order_id]
```

Bad Input

Validate the format before processing.
Return a descriptive error message the agent can relay.

Missing Data

Check existence before returning. A missing key should produce a graceful "not found" response.

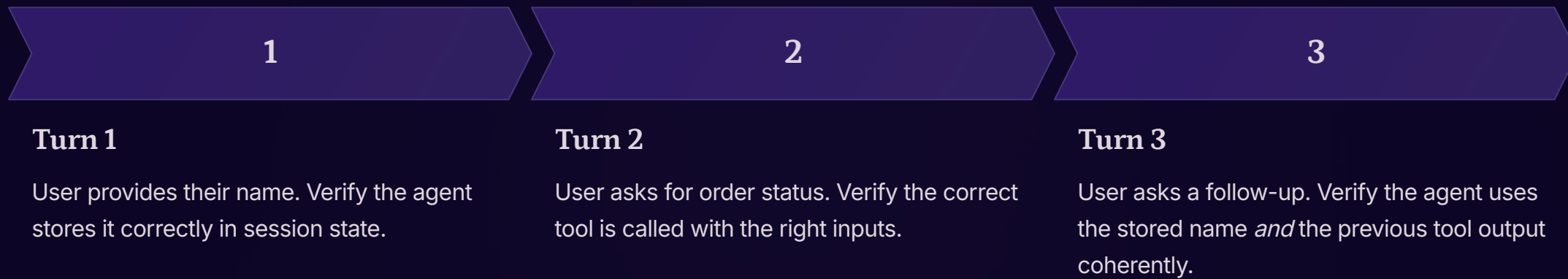
Service Errors

Wrap external calls in try/except. Include a **status** key in your return dict (e.g., **error**, **success**, **pending**) — the LLM uses this as a clear signal about the operation's outcome.

QUESTION 8

How Do You Test Tool Usage and State Together?

Single-turn tests can miss subtle state bugs. **Multi-turn tests** are essential — they verify that memory wiring truly works across a real conversation flow.



⚠ Always assert on **three dimensions**: the tool that was called, the output it returned, and the session state after each turn. Missing any one of these leaves gaps in your test coverage.

QUESTION 9

How Do You Know the Agent Is Ready for the Next Step?

Your agent has graduated from this session when it consistently meets all three readiness criteria — and ADK Web confirms the internals are wired correctly.



Calls Tools Correctly

The agent invokes at least one tool with the right parameters and uses the structured output in its reply.



Stores and Reuses State

Session state is written on new information and correctly read in follow-up turns.



Handles Errors Gracefully

Bad inputs and missing data return clean error dictionaries — the agent explains the issue without exposing internals.

- ✔ Once ADK Web shows tool call details, inputs, outputs, and state changes clearly across turns — you are ready to move from **in-memory state to persistent state**.

What to Explore Next

You've mastered in-memory tools and session state. Here are the natural next questions to guide your continued learning.

Surviving Restarts

What changes when session state must survive a server restart or a new deployment? In-memory state is lost on shutdown — persistent backends solve this.

Moving to Redis

How do you swap `InMemorySessionService` for a Redis-backed store? The interface stays the same — the durability changes entirely.

Richer UI with Structured Output

How does returning well-defined dictionaries from tools unlock richer frontend rendering — such as order cards, status timelines, or dynamic widgets?

Passing Data Between Tools

How do you use the `temp:` prefix in session state to share data between tools within a single agent turn? Temp state is discarded after the invocation, making it ideal for intermediate results.